

Individual assignment cover sheet

Course Name:	Master of Information Systems (Cyber Security)
Unit Number and Name:	MIS5303 Securing Software
Lecturer Name:	Dr Omid Haass
Assignment No.:	Assessment 3 - MediTrack Security Testing Plan

Declaration:

By submitting this declaration, I, the below-named student, declare the following:

- ✓ Except where appropriately acknowledged, this assignment is my own work and has been expressed in my own words.
- ✓ Where I have made use of generative AI software, I have done so appropriately and cited it accurately as per the '**HE-AP19 Generative Artificial Intelligence (AI) Guidelines for Students**' document found in the LMS, See: [HE-AP19 Generative Artificial Intelligence \(AI\) Guidelines for Students.pdf \(apexhe.nsw.edu.au\)](https://apexhe.nsw.edu.au/HE-AP19_Generative_Artificial_Intelligence_(AI)_Guidelines_for_Students.pdf).
- ✓ This assignment in its entirety, or this particular part of it, has not previously been submitted by me or any other person for assessment on this or any other course of study.
- ✓ I have read the Student Academic Misconduct Policy and Procedure found at <https://apexhe.nsw.edu.au/policiesandprocedures/>, and I understand the significance of academic integrity and the consequences of breaching it.

Use of Generative Artificial Intelligence (AI)

Please tick one of the following options:

- ✓ **Declaration 1:** I acknowledge using AI tool(s) to generate content that are included in this work in modified forms.
- ✓ **Declaration 2:** I acknowledge the use of AI tool(s) in the preparation of material that aids the process of completion of this assignment (e.g. for self-study or background reading). No materials are submitted as original work on our part.
- ✓ **Declaration 3:** I did not use any content generated by AI tools in the preparation of this assignment

If your response was **Declaration 1** or **Declaration 2**, please complete the following:

	Student's Response
AI Tool(s) Used (e.g., ChatGPT, Grammarly, DALL·E 2, Pi Assistant, Lumen5, Looka, etc.)	chatgpt
Purpose of Use (Explain specifically what the tool was used for – e.g., idea generation, grammar correction, image generation, summarizing content, etc.)	Fixing format. Comprehending the notion. Idea generation.

List all prompts you have used	
Extent of Modification (Explain if and how you rewrote or modified the AI-generated content. E.g., rephrased, fact-checked, rearranged, quoted suitably, etc.)	
Consider the manners in which your work is different from AI-generated contents.	
How have you modified the generative AI's output?	

Note: All AI usage must be acknowledged and comply with the HE-AP19 Generative Artificial Intelligence (AI) Guidelines for Students. Unacknowledged use of AI tools constitutes academic misconduct.

Student ID Number:	Student Name:
240135	Ashraful Islam

Signed:

Date: 15-Aug-26



Recoverable Signature

X Ashraful

Name

Signed by: 341b0587-a755-4806-a88b-56fe95828d8f

MediTrack — Software Security Testing Plan

A security assurance plan for a health-service clinical notes application

MIS5303 Securing Software — Assessment 3

Prepared by: Ashraful Islam (240135)

Role: Security Lead, MediTrack | Lecturer: Dr Omid Haass

Date: 15-Aug-26

1. Introduction and Scope

MediTrack is an online clinical notes system in Python (Flask) using the SQLite database, with file upload capabilities, running in the Sydney region of Amazon Web Services (AWS) cloud. It allows health allied professions in regional Victoria to account manage, make private clinical notes and upload and share assessments. The application contains sensitive health information for National Disability Insurance Scheme (NDIS) clients, making it subject of the Privacy Act 1988 (Cth) and the thirteen Australian Privacy Principles (APPs) which require an entity to ensure personal information held by them is protected from loss, unauthorised disclosure, unlawful access and misuse (Office of the Australian Information Commissioner [OAIC], n.d.-a). Health information is specifically designated as sensitive information and is assigned the highest level of protection, and the application can be audited by the Department of Health.

The following document is the security testing plan that I have drawn up, being the newly formed security lead. It aims to give the development team, and future auditors, peace of mind that security has not been crammed in at the end of the product lifecycle, but has been designed in, as is common in the existing threat modelling practice (Shostack, 2014). The registration, login, notes and upload modules are complete but have not been formally security-tested and the development team is familiar with Python programming but are new to secure software development. Empirical surveys have shown it is indeed this type of profile where injection and control issues are more prevalent, as a well-functioning program may be completely safe while still being vulnerable to injection attacks (Shar & Tan, 2013). The plan thus includes four integrated areas: threat modelling, the code development of a secure coding standard, a programme to combine manual and automatic testing and an ongoing vulnerability management and reporting process. All recommendations are based on the MediTrack scenario and on Australian legal requirements, as well as on recognised frameworks such as the OWASP Top 10 (around the world: The Open Web Application Security Project [OWASP] Foundation, 2021), the Australian Signals Directorate (ASD) Essential Eight (Australian Cyber Security Centre [ACSC], 2023) and NIST guidance (National Institute of Standards and Technology [NIST], 2017).

2. Threat Modelling

Threat modeling is done first, as it influences all subsequent decisions and is the determinant of what should be protected and what should be avoided to ensure security and privacy by design - it is widely thought of as the most important security and privacy by design technique (Shostack, 2014). I draw a data-flow diagram (DFD) to visualize how information flows in and out of MediTrack and identify where are the trust boundaries and then I write the trust boundaries in and out, using the concepts of the STRIDE taxonomy: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, and Elevation of privilege. STRIDE has shown success with similar health-related systems, such as telehealth systems (Abomhara, et al., 2015) and smart healthcare architectures using Internet-of-Things devices (Nadifi, et al., 2025) and thus, systematic trace of every data flow brings to light real-world threats for every particular scenario, instead of a standard risk checklist..

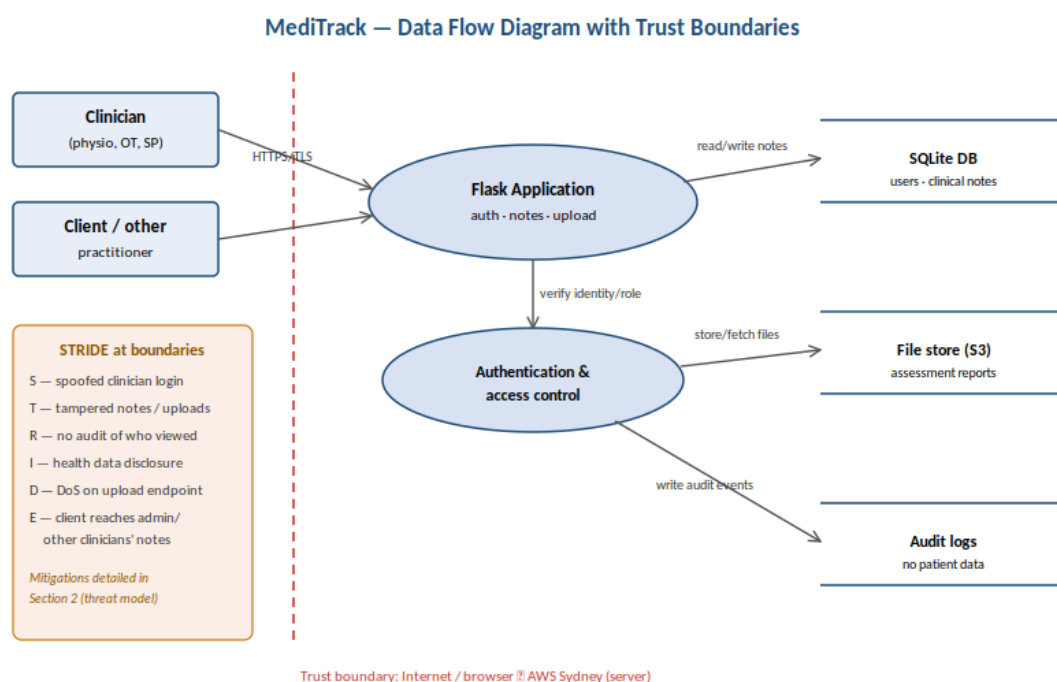


Figure 1: MediTrack data-flow diagram showing trust boundaries and STRIDE exposure.

The situations discussed above depend on the assumption that all traffic across the trust boundary between clinician's browser and the Flask application on AWS is untrusted and it is not until it has been validated. Access to the crown-jewel assets is recorded in a separate audit-log store, which does not contain patient data, and the SQLite database (accounts/clinical notes) and the file store (assessment reports) is where these assets are stored. Realistic risks identified in this scenario and the associated mitigation are below:

STRIDE threat	Realistic attack vector in MediTrack	Impact	Mitigation
Spoofing	Credential stuffing/weak passwords give an attacker the ability to log in as a clinician	Unauthorised access to patient notes	MFA (Essential Eight), bcrypt/argon2 hashing and broken password attempts
Conditionality	Vulnerability to tampering by malicious input alters SQL queries or uploaded report files	Corrupted or falsified clinical records	Parameterised queries, input validation, file integrity checks
Allowing deleted files to be found	Do not record the users who viewed or edited the note of a client	Unable to prove or investigate misuse at audit	Always append notes of access and edit, and file access and download, to an audit log or report
Information disclosure	SQL injection, insecure direct object reference, or an unencrypted backup exposes health data	Notifiable data breach under the Privacy Act	Encryption at rest and in transit, access control, least privilege on AWS
Denial of service	Large or repeated uploads get server resources used up	Clinician in rural areas can't access records	Upload size/rate limits, WAF, AWS auto-scaling
Elevation of privilege	A client or ordinary clinician reaches admin functions or another clinician's notes	Full compromise of confidential records	Server-side role checks on every request, deny-by-default authorisation

These are not generic risks. Information disclosure is weighted highest because a breach of NDIS clients' health data is directly notifiable to the OAIC and to affected individuals under the Notifiable Data Breaches scheme where serious harm is likely (OAIC, n.d.-b), and the sensitivity of Australian health

records has been shown to make disclosure incidents particularly damaging to public trust (Pang et al., 2020). Elevation of privilege is equally severe because clinicians must only ever see their own clients; broken access control of this kind is difficult to detect automatically and typically requires deliberate design and testing to prevent (Vimalachandran et al., 2020). The threat model is a living artefact, revisited whenever a new feature — for example a client self-service portal — changes the data flows, in line with the iterative modelling advocated by Shostack (2014).

3. Secure Coding Standards

Securing development is a new task for the development team so MediTrack has a need for a condensed and strict coding standard that is aligned to the OWASP Top 10 (OWASP Foundation, 2021) and the APPs. All the following are required and will be assessed during code review and automated scanning (Section 4).

- **This includes input validation and output encoding.** The entire input from the users is checked on an allow-list and is considered as untrusted input. All parameters used for accessing the databases are in parameterised queries and data sent to the browser is automatically escaped to prevent the cross site scripting. Injection and XSS are still the two most common and severe vulnerabilities found in the web and input-sanitisation mitigations are the number one defense (Shar & Tan, 2013). This is in response to OWASP A03 Injection.
- **Password handling.** Passwords are NEVER stored in plaintext, nor are they derived with an MD5/SHA-1 julien hash. Passwords are always stored as salted hashes be they bcrypt or argon2 and not as plaintext or with fast hashes like MD5/SHA1. The password policy implementation is based on the NIST SP 800-63B (a minimum length, and passphrases are encouraged, known-breached passwords are prohibited, and the password never needs to be changed) (NIST, 2017). For clinician accounts, multi-factor authentication (MFA) is required, which meets a core Essential Eight control (ACSC, 2023) and aligns with APP 11 (OAIC, n.d.-a).
- **Secure data storage.** AWS KMS-managed keys are used to encrypt clinical notes and all traffic is served over TLS 1.2 or updated. All files uploaded are checked by extension, MIME-type against a list of so-called whitelists, stored with server-generated names outside the webroot, and limited in size to prevent a file upload of malware or path traversal.
- **Authentication & authorization.** A clinician may access only their own clients' records using denylist by default checks against the authorisation at the server level for each request. AWS Identity and Access Management (IAM) roles are specified with the least privilege that any given component requires, thereby reducing the scope of damage from any compromises suffered.
- **Guaranteed handling and recording of errors.** If there are errors, they return generic comments to the users, and vast details are logged server-side. The logging system is used in compliance with the Privacy Act, audit evidence will be backward compatible and the debug mode and logging of passwords and session tokens/wallet will not be allowed in production.
- **Requirements - Secrets and dependency management.** Secret keys or database credentials are never hard coded, but loaded from environment variables or AWS Secrets Manager. To ensure the safety of vulnerable components, which are also part of the Top 10 risk category (OWASP Foundation, 2021), vulnerable third-party components are also selected and continuously monitored for known vulnerabilities.

4. Security Testing Plan

Security testing is a multi phase activity during the software lifecycle, both automated and manual testing methods, which are executed both cheaply and continuously and independently, with the manual security testing targeting only those portions of software that are not adequately covered in the automated methods. This layered approach is based on the OWASP Web Security Testing Guide (OWASP Foundation, 2022) and the literature fact that there is no single testing technique that could be used alone (Shar & Tan, 2013). Each activity in the programme below indicates the approach to be taken, the tools, what scope, when and by whom.

Activity	Tool / approach	Scope	Timing	Responsible
Static analysis (SAST)	Bandit, Semgrep	All Python source	Every commit (CI pipeline)	Developers / DevOps
Dependency scanning	pip-audit, GitHub Dependabot	Third-party packages	Every commit + weekly	DevOps
Secret scanning	git-secrets / truffleHog	Repository	Every commit	DevOps
Dynamic analysis (DAST)	OWASP ZAP	Running app in staging	Each sprint / pre-release	Security lead
Manual code review	Peer review vs this standard	Auth, notes, upload modules	Each pull request	Senior developer
Penetration testing	Manual, OWASP Testing Guide	Full application	Before go-live, then annually	External tester / security lead

Automated SAST (Bandit and Semgrep) is used to identify patterns of access that may pose a security threat, such as Shell Command Execution, Weak Hashing, and Raw SQL, at an early stage and low cost, on each and every commit. In DAST with OWASP ZAP, the running application is exercised in a dedicated 'staging environment' to seek runtime vulnerabilities such as injection attacks, broken authentication, and misconfiguration, which are followed by the methodology described in OWASP Testing Guide (OWASP Foundation, 2022). But, automated tools alone are not enough: they cannot reliably detect broken access control – for instance a clinician looking at another clinician's notes, as repeatedly noted for health systems with sensitive records (Vimalachandran et al, 2020). Manual code review and penetration testing are then targeted at the authorisation logic, session handling and upload workflow. Testing itself never carries risk of privacy breach, as the data in all cases is synthetic data (not real patient), which is an ethical and legal requirement given the sensitivity of the data (Pang et al., 2020).

5. Vulnerability Management and Reporting

Keeping vulnerabilities is only valuable when there is a systematic methodology to prioritize, patch, validate and disclose vulnerabilities. This follows the continuous five phases that can be correlated to the Essential Eight's patch-applications control and its focus on remediation of exploitable weaknesses at the right time (ACSC, 2023).

Stage	Action
1. Identify	Findings from SAST, DAST, code review, penetration tests and dependency scans will be written on a central tracker with evidence
2. Triage / prioritise	Each finding is given a severity rating and an exploitability (CVSS). Anytime there is patient data involved, it is imperatively treated as critical
3. Remediate	Owners fix within specified SLAs critical: within 48 hours; high: within one week with patches or compensating controls
4. Retest	A retest is performed again to insure that the fix is correct and there is no regression before closure of this ticket

Stage	Action
5. Report / document	Status is reported to management; record meets accountability requirement of the Privacy Act and audits for the Department of Health.

Reporting is directly related to compliance. The Notifiable Data Breaches scheme under the Privacy Act sets out requirements for the entity to consider the circumstances of the vulnerability when deciding whether there has been unauthorised access to information, and if there has been, inform the OAIC and them of the incident if there is likely to be serious harm (OAIC, n.d.-b). It will give auditors the evidence they expect and enable a record of what actions or outcomes MediTrack has taken to adhere to APP 11's requirement to take reasonable steps to protect personal information (OAIC, n.d.-a). It is highly feasible that data breaches could quickly cause loss of consumer trust (Pang et al., 2020), so health data reporting would also serve as a safeguard of the NDIS provider that is implementing the application in terms of reputation.

6. Integration and Conclusion

The four areas are not a list that needs action taken but are a part of one cycle of assurance. The secure coding standards prevent risks from occurring at source (OWASP Foundation, 2021; NIST, 2017); the testing programme identifies the risks that make it through (OWASP Foundation, 2022); and the vulnerability-management process ensures risks are addressed and verified and reported against compliance obligations (ACSC, 2023; OAIC, n.d.-b). Security-by-design: the process continues and is fueled by each step influencing the next, state-of-the-art or new findings influencing the threat model, auditing requirements influencing what the process should be documenting, etc., as MediTrack's. Contextual basis for the design is considered throughout—including how to keep the health information of MediTrack's NDIS clients confidential; adherence to the Privacy Act 1988 and the APPs; and the limitations and accessibility of the AWS Sydney location, in addition to the Essential Eight controls. This plan will give the development team clear, specific advice and reassure the next generation of auditors that security has been baked into the back of the MediTrack reactor, reducing the chance of a notifiable breach and any of the possible repercussions.

References

- Abomhara, M., Gerdes, M., & Koien, G. M. (2015). A STRIDE-based threat model for telehealth systems. *Norsk Informasjonssikkerhetskonferanse (NISK)*, 8(1), 82-96.
- Australian Cyber Security Centre. (2023). *Essential Eight maturity model*. Australian Signals Directorate. <https://www.cyber.gov.au/resources-business-and-government/essential-cyber-security/essential-eight>
- Hollo, Z., & Martin, D. E. (2023). An equitable approach to enhancing the privacy of consumer information on My Health Record in Australia. *Health Information Management Journal*, 52(3), 190-197. <https://doi.org/10.1177/18333583211019764>
- Nadifi, Z., Ouaisa, M., Ouaisa, M., Alhyan, M., & Kartit, A. (2025). STRIDE-based threat modeling and risk assessment framework for IoT-enabled smart healthcare systems. *International Journal of Online and Biomedical Engineering*, 21(9), 63-80. <https://doi.org/10.3991/ijoe.v21i09.55517>
- National Institute of Standards and Technology. (2017). *Digital identity guidelines: Authentication and lifecycle management (SP 800-63B)*. U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-63b>
- Office of the Australian Information Commissioner. (n.d.-a). *Australian Privacy Principles*. Australian Government. <https://www.oaic.gov.au/privacy/australian-privacy-principles>
- Office of the Australian Information Commissioner. (n.d.-b). *Notifiable Data Breaches scheme*. Australian Government. <https://www.oaic.gov.au/privacy/notifiable-data-breaches>
- OWASP Foundation. (2021). *OWASP Top 10:2021*. <https://owasp.org/Top10/>
- OWASP Foundation. (2022). *Web Security Testing Guide (v4.2)*. <https://owasp.org/www-project-web-security-testing-guide/>
- Pang, P. C. I., McKay, D., Chang, S., Chen, Q., Zhang, X., & Cui, L. (2020). Privacy concerns of the Australian My Health Record: Implications for other large-scale opt-out personal health records. *Information Processing & Management*, 57(6), 102364. <https://doi.org/10.1016/j.ipm.2020.102364>
- Parliament of Australia. (1988). *Privacy Act 1988 (Cth)*. Federal Register of Legislation. <https://www.legislation.gov.au/C2004A03712>
- Shar, L. K., & Tan, H. B. K. (2013). Predicting SQL injection and cross site scripting vulnerabilities through mining input sanitization patterns. *Information and Software Technology*, 55(10), 1767-1780. <https://doi.org/10.1016/j.infsof.2013.04.002>
- Shostack, A. (2014). *Threat modeling: Designing for security*. Wiley.
- Sivakumar, N. R. (2025). Kademia hash snow ablation resource optimized stride scheduling for mobile computing services in healthcare sector. *Scientific Reports*, 15, Article 15913. <https://doi.org/10.1038/s41598-025-15913-w>
- Vimalachandran, P., Zhang, Y., Cao, J., Sun, L., & Yong, J. (2020). Improving accessibility of the Australian My Health Records while preserving privacy and security of the system. *Health Information Science and Systems*, 8, Article 31. <https://doi.org/10.1007/s13755-020-00126-4>